Phase 2 - Week 6 - Lesson Day 1

Decision Trees

Today's Goal

Last week you learned Linear and Logistic Regression. Today you meet a very different kind of model: the **Decision Tree**. It makes decisions by asking a series of simple yes/no questions, just like the game "20 Questions".

- Understand how a decision tree makes a prediction
- Train a decision tree with Scikit-learn
- Read a tree diagram
- Understand overfitting (memorizing) and how max_depth helps

1. What Is a Decision Tree?

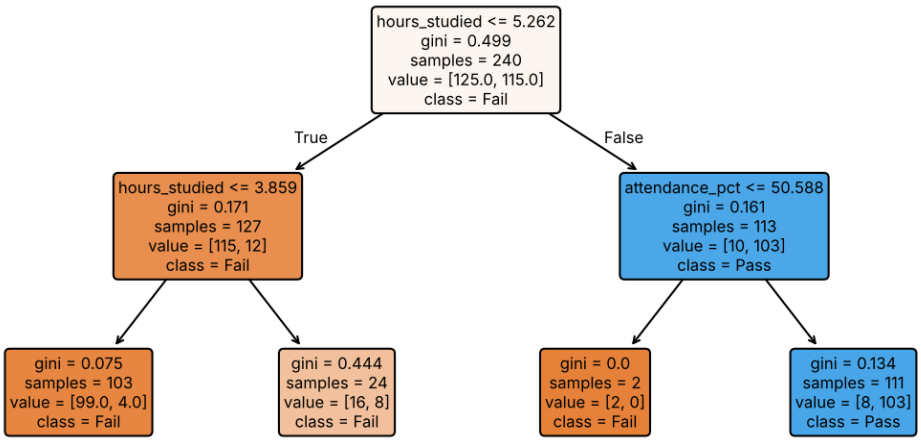
> **Simple definition:** A decision tree is a model that asks yes/no questions about your data, one after another, until it reaches an answer. Each question splits the data into two smaller groups.

Example: to guess if a student passes, the tree might ask "Did they study more than 5 hours?" If yes, it asks another question. If no, it asks a different question. At the end of each path is the final answer: Pass or Fail.

You already know this kind of thinking. It is just a chain of **if / else** statements, but the computer finds the best questions to ask by itself, using the training data.

2. Reading a Tree Diagram

A Decision Tree (depth 2): questions lead to an answer



Each box in the picture above has the same parts:

Part of the box	What it means
First line (a question)	The yes/no question. If True, go LEFT. If False, go RIGHT.
gini	How mixed the group is. 0 means everyone in the box is the same (pure). Higher means more mixed.
samples	How many training rows reached this box.
value = [a, b]	How many were Fail (a) and how many were Pass (b).
class	The answer the tree gives if you stop at this box. The color matches it (orange = Fail, blue = Pass).

Follow the right side: a student studied more than 5.26 hours, and attendance was above 50.6, so the tree lands in the box with value = [8, 103]. That means 103 similar students passed and only 8 failed, so the tree predicts **Pass**.

3. Training a Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)            # learn the questions from the training data

predictions = tree.predict(X_test)    # predict on data the tree has not seen
print(tree.score(X_test, y_test))     # accuracy on the test data
```

It is the same pattern as last week: **.fit()** to learn, **.predict()** to predict, and **.score()** to check. Only the model name changed.

4. Overfitting and max_depth

> **Simple definition:** Overfitting means the model memorizes the training data instead of learning the real pattern. It scores great on data it has already seen, but poorly on new data.

A decision tree is very easy to overfit. If you let it ask unlimited questions, it keeps splitting until every single training row has its own box. That is memorizing, like a student who memorizes the exact exam answers without understanding the subject.

max_depth limits how many questions in a row the tree can ask. A small max_depth (like 3) forces the tree to stay simple and learn only the big, real patterns.

```
# No limit: the tree can memorize
memorizer = DecisionTreeClassifier(random_state=42)
memorizer.fit(X_train, y_train)
print(memorizer.score(X_train, y_train))   # 1.0 -> perfect on training data
print(memorizer.score(X_test, y_test))     # lower -> worse on new data

# Limit the depth: the tree stays simple
simple = DecisionTreeClassifier(max_depth=3, random_state=42)
```

How to spot overfitting: if the training score is much higher than the test score, the model is memorizing. You want the two scores to be close.

Today's Practice

Open **day-1-decision-trees.ipynb**. All answers are already solved, with simple code and comments.

Practice Questions (5 total - all solved)

1. Prepare the data

Create the student dataset and split it into training (80%) and test (20%).

2. Train a tree

Train a DecisionTreeClassifier with max_depth=3.

3. Predict

Predict for the first 10 test students and print them next to the real answers.

4. Spot overfitting

Train a tree with no max_depth. Print its training score and test score and explain the gap.

5. Draw the tree

Draw your max_depth=3 tree with plot_tree().

Once you and Halimo finish, move to Lesson Day 2 (Random Forests).